

Desarrollo de Sistemas Móviles

Sesión 6: Base de Datos Locales
Room 2.6+, DataStore, WorkManager y Firebase

Comenzamos en 8:40am

Al finalizar la clase, serás capaz de:

- **Room 2.6+:** Implementar persistencia local con consultas reactivas usando `Flow<List<T>>`.
- **DataStore:** Reemplazar `SharedPreferences` con `Preferences DataStore` y `Proto DataStore`.
- **WorkManager 2.9+:** Programar tareas en segundo plano que sobreviven la muerte de la app.
- **Firestore Auth:** Gestionar usuarios con `callbackFlow` reactivo. Comparar con JWT propio.
- **Firestore vs Realtime DB:** Elegir la solución correcta para cada caso de uso en 2026.

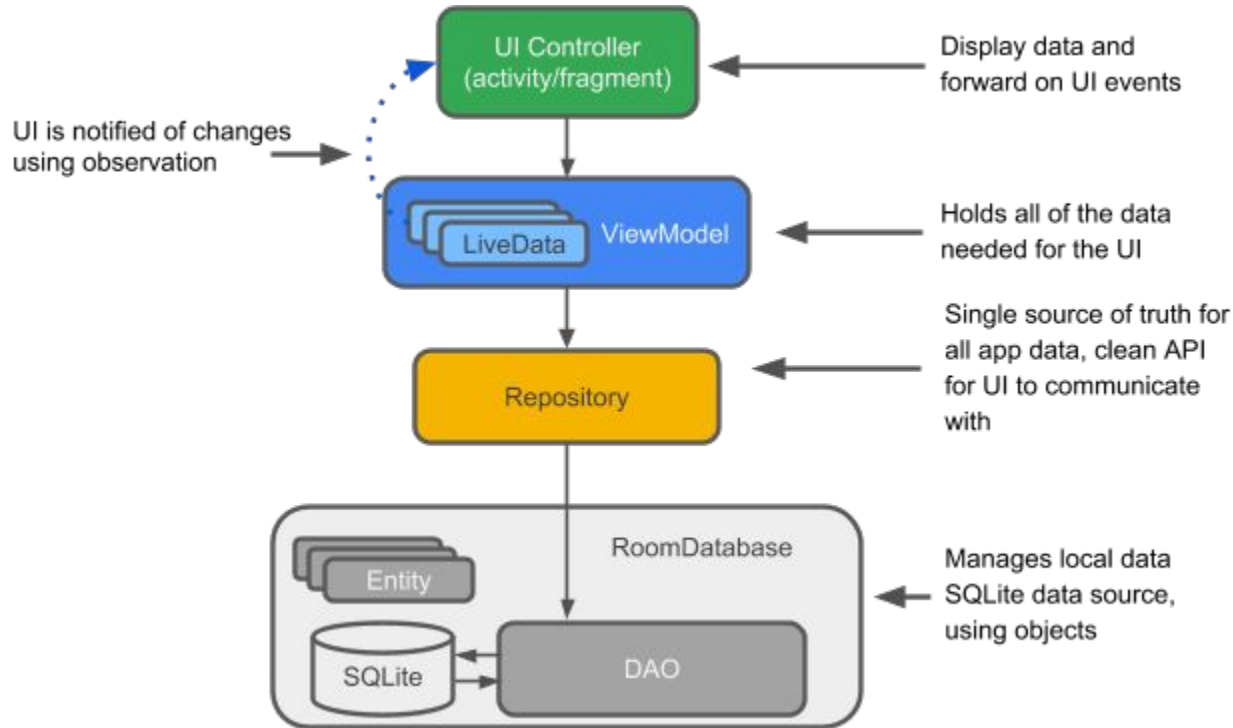
¿Por qué Room en lugar de SQLite directo?

- **Verificación en compilación:** Las consultas SQL se validan en tiempo de compilación, no en runtime.
- **Reactividad nativa:** `Flow<List<T>>` integrado — la UI se actualiza automáticamente.
- **Data classes automáticas:** Sin manejo manual de Cursor. Los resultados se mapean directo a objetos.
- **Migraciones versionadas:** Scripts de migración controlados, nunca destructivos en producción.
- **TypeConverters:** Soporte para tipos complejos (List, Date, Enum) con conversores personalizados.
- **Testing fácil:** Base de datos in-memory para tests unitarios sin depender del dispositivo.



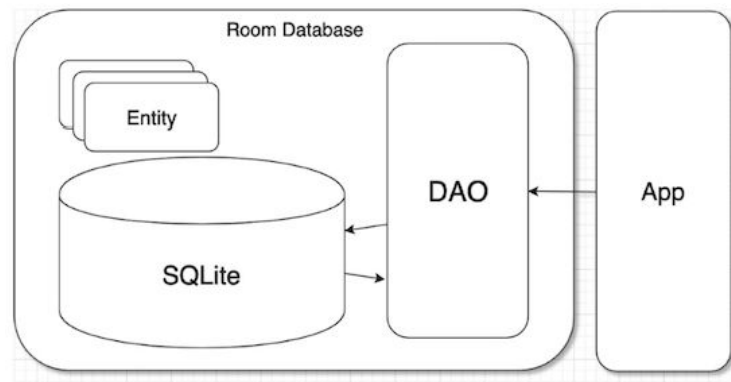
android

**Room database
persistence Library**



Room se compone de tres capas que trabajan juntas:

- **@Entity:** Representa una tabla en la base de datos. Cada campo es una columna. Usa `@PrimaryKey`, `@ColumnInfo` y `@Embedded`.
- **@Dao (Data Access Object):** Interfaz con las consultas. `@Query`, `@Insert`, `@Update`, `@Delete`, `@Upsert`. Las `suspend fun` y `Flow<T>` se integran nativamente.
- **@Database:** Punto de entrada. Extiende `RoomDatabase`. Define las entities y la versión. Singleton con doble verificación (`INSTANCE + synchronized`).
- **TypeConverters:** Para tipos no soportados por SQLite (`List<String>`, `LocalDate`). `@TypeConverter` en/desde `String` o `Long`.



[그림 4-3] Room 구성 요소 간 관계

DemoData: AppDatabase gestiona 4 entidades: GpsGoogle, GpsSensors, Media y Audio en `fleet.db`.

```
// 1. ENTITY
@Entity(tableName = "products")
data class ProductEntity(
    ... @PrimaryKey val id: Int,
    ... @ColumnInfo(name = "product_name") val nombre: String,
    ... val precio: Double,
    ... val categoria: String,
    ... val disponible: Boolean = true,
    ... @ColumnInfo(name = "created_at") val creadoEn: Long = System.currentTimeMillis()
)
```

```
// TypeConverter para tipos no soportados
class Converters {
    ... @TypeConverter
    ... fun fromList(v: List<String>): String = v.joinToString(",")
    ... @TypeConverter
    ... fun toList(v: String): List<String> = v.split(",")
}
```

```
// 2. DAO
@Dao
interface ProductDao {
    ... @Query("SELECT * FROM products ORDER BY product_name ASC")
    ... fun getAll(): Flow<List<ProductEntity>> ... // reactivo

    ... @Query("SELECT * FROM products WHERE product_name LIKE '%'| :q | '%'")
    ... fun search(q: String): Flow<List<ProductEntity>>

    ... @Upsert // insert OR update
    ... suspend fun upsert(product: ProductEntity)

    ... @Delete
    ... suspend fun delete(product: ProductEntity)
}
```

```
// 3. DATABASE
@Database(entities = [ProductEntity::class], version = 1, exportSchema = true)
@TypeConverters(Converters::class)
abstract class AppDatabase : RoomDatabase() {
    ... abstract fun productDao(): ProductDao

    ... companion object {
        ... @Volatile private var INSTANCE: AppDatabase? = null

        ... fun getInstance(context: Context) = INSTANCE ?: synchronized(this) {
            ... INSTANCE ?: Room.databaseBuilder(
            ... context.applicationContext, AppDatabase::class.java, "app_database"
            ... ).addMigrations(MIGRATION_1_2).build().also { INSTANCE = it }
            ... }
        ... }
    }
}
```

Regla de oro: NUNCA **fallbackToDestructiveMigration()** en producción.

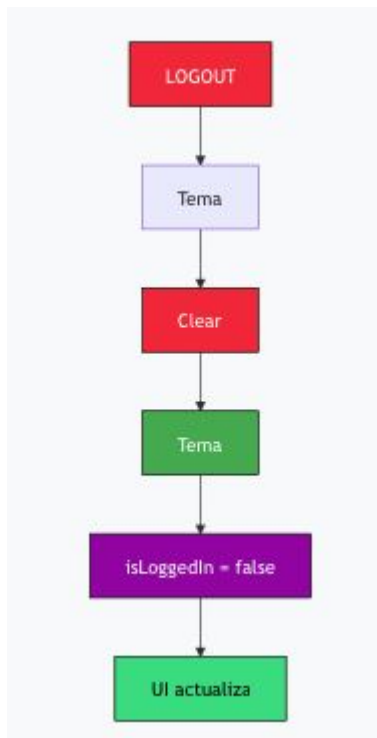
- **Migration(oldVer, newVer):** Implementa override fun migrate(db: SupportSQLiteDatabase) con los ALTER TABLE o CREATE TABLE necesarios.
- **Registro obligatorio:** .addMigrations(MIGRATION_1_2) en el builder. Room ejecuta la cadena de migraciones automáticamente.
- **Campos nullable en produccion:** latitude: Double? en GpsGoogleEntity evita confundir “sin señal GPS” con coordenada (0.0, 0.0) en el Golfo de Guinea.
- **Ejercicio DemoData:** DemoDataDatabase está en version=3 con migración destructiva. Tarea: escribir MIGRATION_2_3 que agrega isSynced INTEGER NOT NULL DEFAULT 0 a las 4 tablas.
- **exportSchema = true:** Genera archivos JSON con el historial del esquema. Imprescindible para auditorías y testing de migraciones.

SharedPreferences era la herramienta original de Android para **guardar pequeñas cantidades de datos simples** en forma de pares **clave-valor**.

Fue la solución estándar durante muchos años antes de que Google recomendara DataStore.

Problemas de **SharedPreferences** que **DataStore** soluciona:

- **Bloqueo del hilo principal:** Las operaciones de lectura/escritura pueden generar ANR en dispositivos lentos con archivos grandes.
- **Sin null-safety:** `getBoolean()`, `getString()` pueden retornar null inesperadamente sin advertencia del compilador.
- **Sin soporte para coroutines:** No hay `Flow<T>` integrado. Hay que usar `OnSharedPreferenceChangeListener` (callback hell).
- **Sin verificación de tipos:** Se puede escribir un `Boolean` y leerlo como `String` sin error en compilación.
- **DataStore Preferences:** API Flow-based, type-safe, suspend. Lee con `dataStore.data.map{}`, escribe con `dataStore.edit{}`.
- **Proto DataStore:** Para esquemas complejos con Protobuf. Tipado fuerte y migración de esquema explícitamente definida.



```
class SessionManager(private val context: Context) {  
    private companion object {  
        val KEY_IS_LOGGED_IN = booleanPreferencesKey("is_logged_in")  
        val KEY_ACCESS_TOKEN = stringPreferencesKey("access_token")  
        val KEY_DARK_MODE = booleanPreferencesKey("dark_mode")  
    }  
  
    val isLoggedIn: Flow<Boolean> = context.sessionDataStore.data  
        .map { prefs -> prefs[KEY_IS_LOGGED_IN] ?: false }  
  
    // Caso especial: logout que PRESERVA el tema  
    suspend fun logout() {  
        context.sessionDataStore.edit { prefs ->  
            val currentTheme = prefs[KEY_DARK_MODE] // leer antes de borrar  
            prefs.clear() // borra tokens + sesión  
            if (currentTheme != null) prefs[KEY_DARK_MODE] = currentTheme  
        }  
    }  
  
    // Clave: edit{} recibe MutablePreferences -> puedes leer y escribir  
    // en la misma transacción atómica.
```

¿Cuándo usar WorkManager?:

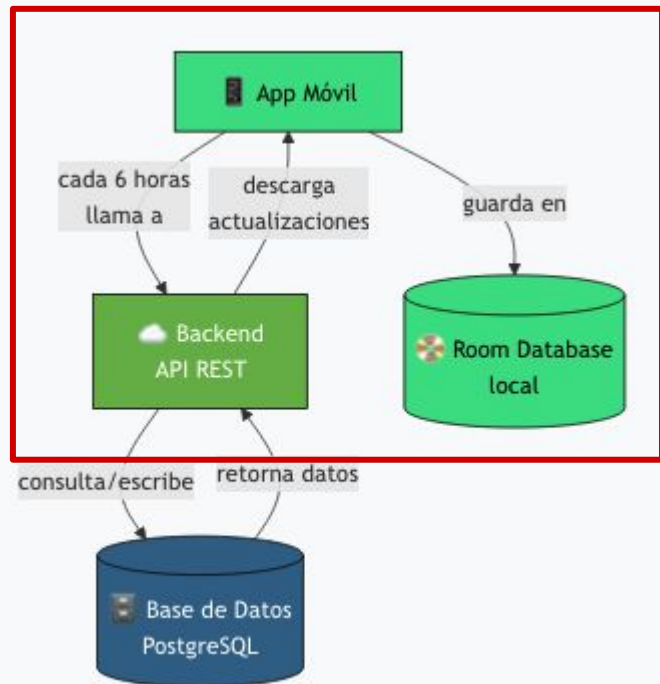
Para tareas diferibles que **DEBEN** completarse eventualmente.

- **Sincronizar datos en background:** Enviar logs de analytics, subir fotos, limpiar datos temporales.
- **Sobrevive la muerte de la app:** Si el proceso muere, WorkManager retoma el trabajo al reiniciar.
- **Constraints declarativos:** `.setRequiredNetworkType(CONNECTED)`, `.setRequiresBatteryNotLow(true)`, `.setRequiresCharging(false)`.
- **Worker vs CoroutineWorker:** `Worker.doWork()` para trabajo síncrono/bloqueante. `CoroutineWorker` con `suspend doWork()` para llamadas a Room/Retrofit.
- **ExistingPeriodicWorkPolicy.KEEP:** No crea una segunda instancia si el trabajo ya está encolado. Evita duplicados.
- **NO usar para:** Tareas inmediatas (usar `viewModelScope` + `coroutines`) o alarmas precisas (usar `AlarmManager` + `BroadcastReceiver`).

```
class SincronizacionWorker(ctx: Context, params: WorkerParameters)
    : CoroutineWorker(ctx, params) {}

    override suspend fun doWork(): Result {
        // corre en Dispatchers.IO
        return try {
            val datos = api.obtenerActualizaciones()
            database.guardarTodos(datos)
            Result.success()
        } catch (e: Exception) {
            if (runAttemptCount < 3) Result.retry() else Result.failure()
        }
    }
}

// Programar trabajo periódico cada 6 horas
WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "sincronizacion_periodica",
    ExistingPeriodicWorkPolicy.KEEP,
    PeriodicWorkRequestBuilder<SincronizacionWorker>(6, TimeUnit.HOURS)
        .setConstraints(Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .setRequiresBatteryNotLow(true).build())
        .build()
)
```



Métodos de autenticación disponibles:

- **Email + Contraseña:** Para apps propias sin SSO. `createUserWithEmailAndPassword()` y `signInWithEmailAndPassword()`.
- **Google Sign-In:** Mayor UX — un clic, sin contraseñas. Requiere SHA-1 en Firebase Console y `GoogleSignInClient`.
- **Autenticación anónima:** Permite uso sin registro. Luego se puede vincular con una cuenta real via `linkWithCredential()`.
- **Phone SMS OTP:** Para verificación por celular. `PhoneAuthProvider` + SMS enviado por Firebase.
- **Firebase Security Rules:** Proteger con `request.auth != null && request.auth.uid == userId`. Nunca dejar reglas abiertas.
- **DemoData contrast:** DemoData usa JWT propio con Retrofit + SessionManager (DataStore). Comparación: `callbackFlow` de Firebase vs `Flow` de DataStore.

```
class AuthRepositoryImpl : AuthRepository {
    ✦ private val auth = Firebase.auth

    ---// Estado del usuario como Flow reactivo
    ---val currentUser: Flow<FirebaseUser?> = callbackFlow {
    ---|    ---val listener = FirebaseAuth.AuthStateListener { trySend(it.currentUser) }
    ---|    ---auth.addAuthStateListener(listener)
    ---|    ---awaitClose { auth.removeAuthStateListener(listener) }
    ---|    }

    ---suspend fun signIn(email: String, password: String): Result<FirebaseUser> =
    ---|    ---runCatching {
    ---|    ---|    ---auth.signInWithEmailAndPassword(email, password).await().user
    ---|    ---|    ---?: throw Exception("Error de autenticación")
    ---|    ---}

    ---fun signOut() = auth.signOut()
}

// DemoData usa JWT propio: Flow de DataStore en vez de callbackFlow de Firebase.
// Misma idea reactiva, diferente fuente de datos.
```

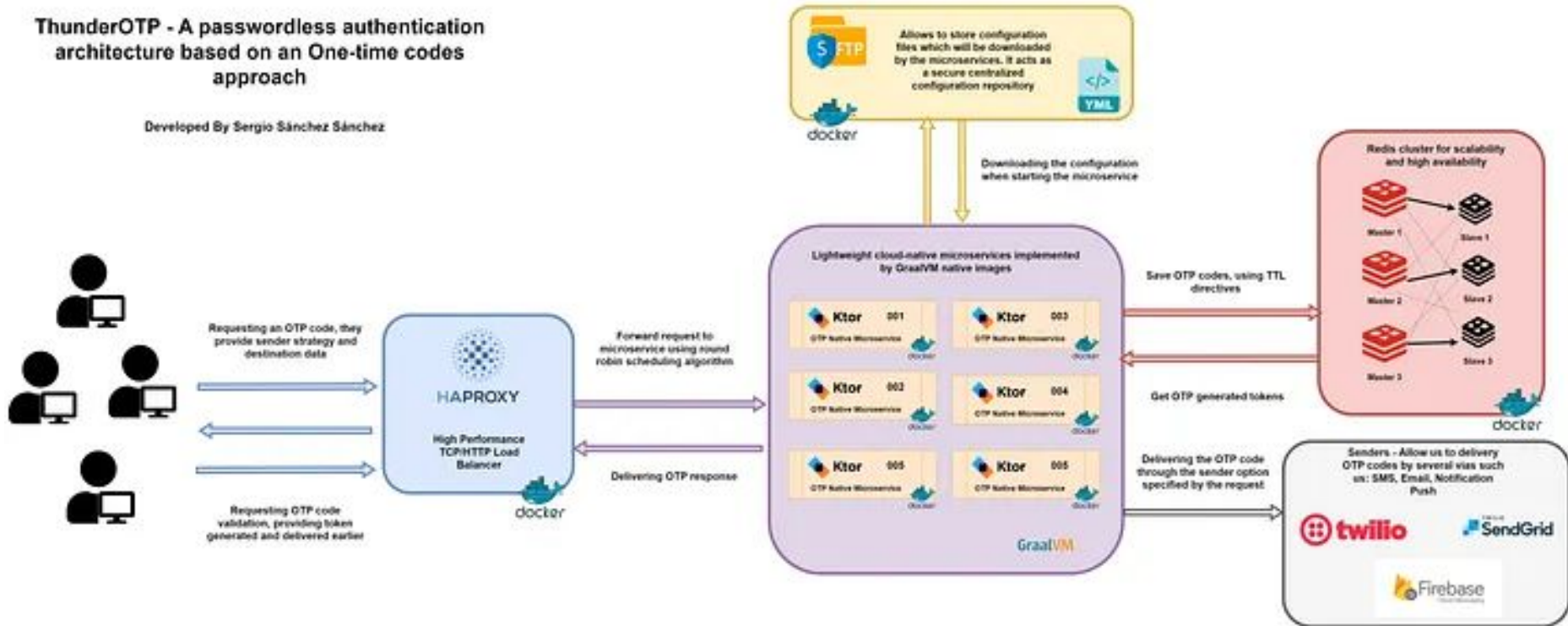
Types of Authentication



Authentication

ThunderOTP - A passwordless authentication architecture based on an One-time codes approach

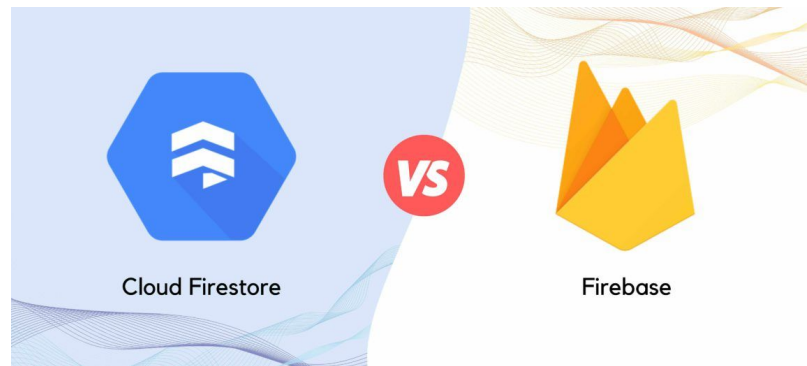
Developed By Sergio Sánchez Sánchez



<https://medium.com/better-programming/a-passwordless-authentication-architecture-based-on-a-one-time-code-approach-2ef30d0f25d8>

Recomendación 2026: Firestore como solución primaria para proyectos nuevos.

- **Estructura de datos:** Firestore: documentos + colecciones flexibles. Realtime DB: árbol JSON rígido — difícil de reestructurar después.
- **Consultas:** Firestore soporta filtros compuestos, ordenamiento y paginación. Realtime DB: solo por clave/valor básico.
- **Offline support:** Firestore: caché automática completa. Realtime DB: soporte básico con `persistDiskCache()`.
- **Escalabilidad:** Firestore: serverless, escala automáticamente. Realtime DB: tiene límites de conexiones simultáneas.
- **Precio:** Firestore cobra por lectura/escritura/almacenamiento. Realtime DB por GB transferido. Firestore es predecible.
- **Realtime DB legacy:** Solo para proyectos existentes. Nuevos proyectos siempre Firestore.



Lab

11:20 am

por favor vamos al proyecto de la semana pasada

Gracias

